

CI/CD Pipeline & Automated Error Diagnosis

A Flask/Python Implementation Guide - August 2026

Overview

This document describes a comprehensive CI/CD testing pipeline and automated error diagnosis system for Flask/Python web applications. The system provides:

- 1. **Three-Layer Testing Pipeline** - Fast TDD, integration tests, and full smoke tests
- 2. **Automated Error Capture** - Production errors logged with full context
- 3. **Local AI Diagnosis** - Claude Code analyzes errors with full codebase context
- 4. **TDD Fix Workflow** - Every fix includes a regression test

Key Benefits

Benefit	Description
Fast Feedback	Quick tests run in <5 seconds for TDD workflow
Full Coverage	Smoke tests exercise every feature before deployment
Automatic Capture	Production errors captured with request context and traceback
AI-Assisted Fixes	Claude Code diagnoses with full codebase understanding
Regression Prevention	Every bug fix creates a permanent test

Part 1: Three-Layer Testing Pipeline

Layer Overview

Layer	Name	Speed	Purpose	When to Run
1	Quick	~5s	TDD development	Every code change
2	Integration	~60s	Feature verification	Before commits
3	Smoke	5-15min	Full site coverage	Before deployment

Layer 1: Quick Tests

Quick tests run without database connections and verify core logic.

Characteristics

- No database required
- No external dependencies
- Tests pure functions and utilities
- Marked with `@pytest.mark.quick`

Running Quick Tests

```
python run_tests.py --quick
```

Example Quick Test

```
import pytest
|
@pytest.mark.quick
class TestDateUtils:
    def test_format_game_date(self):
        from app.utils.dates import format_game_date
        result = format_game_date(datetime(2026, 9, 15, 18, 30))
        assert result == "Sep 15, 6:30 PM"
```

Layer 2: Integration Tests

Integration tests verify features work correctly with the database.

Characteristics

- Requires database connection
- Uses factory fixtures for test data
- Tests routes and database operations
- Automatic cleanup after each test

Running Integration Tests

```
python run_tests.py # All integration tests
python run_tests.py auth # Just auth test
python run_tests.py fields games # Multiple test files
```

Factory Fixtures

Factory fixtures create test data with automatic cleanup:

```
# In conftest.py
@pytest.fixture
def field_factory(app):
    """Factory for creating test fields."""
    created = []

    def _create(name=None, **kwargs):
        with app.app_context():
            field = Field(
                location_title=name or f'Test Field {uuid.uuid4().hex[:8]}'
                active=1,
                **kwargs
            )
            db.session.add(field)
            db.session.commit()
            created.append(field.ID)
            return field

    yield _create

    # Cleanup
    with app.app_context():
        for field_id in created:
            Field.query.get(field_id).active = 0
        db.session.commit()
```

Using Factories in Tests

```
def test_field_creation(field_factory, scheduler_client):
    """Test creating a new field."""
    field = field_factory('Test Field', is_owned=1)
    |
    response = scheduler_client.get(f'/fields/{field.ID}')
    assert response.status_code == 200
    assert b'Test Field' in response.data
```

Layer 3: Smoke Tests

Smoke tests exercise every feature from an inventory file.

Feature Inventory (tests/inventory.yaml)

```
features:
  - name: Authentication
    category: auth
    routes:
      - path: /auth/login
        method: GET
        expect: 200
      - path: /auth/login
        method: POST
        data: {email: "test@example.com", password: "test123"}
        expect: 302
    |
  - name: Field Management
    category: fields
    requires_auth: scheduler
    routes:
      - path: /fields/
        method: GET
        expect: 200
        contains: ["Field Management", "Add Field"]
```

Running Smoke Tests

```
python run_tests.py --full
# or
python scripts/smoke_test.py
```

Pre-Push Validation

A git hook runs validation before every push:

Setup

```
git config core.hooksPath .githubhooks
chmod +x .githubhooks/pre-push # Linux/Mac
```

What It Checks

- 1. Quick tests pass
- 2. No syntax errors
- 3. No obvious security issues
- 4. Integration tests pass (optional)

Part 2: Error Capture System

Tier System

Errors are classified into tiers for appropriate handling:

Tier	Name	Behavior	Examples
I	Critical	Immediate alert	Database failures, 500 errors, auth issues
II	Digest	Periodic summary	Analytics failures, minor validation errors

Database Schema

```
CREATE TABLE sdll_app_errors (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
  -- Classification  
  tier SMALLINT NOT NULL DEFAULT 2,  
  context VARCHAR(100) NOT NULL,  
  error_type VARCHAR(100) NOT NULL,  
  error_message TEXT NOT NULL,  
  traceback TEXT,  
  -- Request context  
  request_method VARCHAR(10),  
  request_path VARCHAR(500),  
  request_user_agent VARCHAR(500),  
  user_id INT,
```

```
|
|
| -- Status
| notified BOOLEAN DEFAULT FALSE,
| resolved BOOLEAN DEFAULT FALSE,
| resolved_at DATETIME,
| resolved_by INT,
| error_hash VARCHAR(64)
|)
|)
```

Error Logging Utilities

Basic Usage

```
from app.utils.errors import log_error, log_tier1, log_tier2
|
| # Auto-classify based on context
| log_error('page_view_tracking', exception, request)
|
| # Force Tier I (critical)
| log_tier1('database_connection', exception, request)
|
| # Force Tier II (digest)
| log_tier2('analytics', exception, request)
```

Safe Decorators

```
from app.utils.errors import safe_tracking
|
| @safe_tracking
| def log_page_view(request, user_id):
|     """This will never crash - errors are logged silently."""
|     page_view = PageView(...)
|     db.session.add(page_view)
|     db.session.commit()
```

Global Error Handler

```
# In app/ init .py
from app.utils.errors import register_global_handler
|
| def create_app(config_name=None):
|     app = Flask(__name__)
|     # ... setup ...
|     register_global_handler(app)
|     return app
```

This catches all unhandled exceptions and logs them as Tier I errors.

Part 3: Automated Error Diagnosis

Architecture

```

PRODUCTION (Railway)                                LOCAL (Developer Machine)
=====
|
500 Error Occurs
↓
Global handler catches it
↓
Stored in sdll_app_errors > Scheduled task polls DB
↓
every 5 minutes
Alert sent (optional) ↓
Error exported to
errors/pending/error_N.md
↓
Developer runs Claude Code
$ claude "Diagnose error N"
↓
Claude reads error + source
Creates reproducing test
Implements fix
Runs all tests
Asks for approval
Commits fix + test

```

File Structure

```

project/
├── errors/
│   ├── .gitignore # Ignore error files
│   ├── pending/ # Errors awaiting diagnosis
│   │   ├── error_1.md
│   │   └── error_1.json
│   ├── diagnosed/ # Completed diagnoses
│   ├── .state.json # Rate limiting state
│   └── scripts/
│       ├── poll_errors.py # Polls DB, exports errors
│       ├── diagnose_error.py # Helper to view/manage errors
│       └── setup_error_poll_task.bat # Windows task setup
├── tests/
│   └── test_regressions.py # Regression tests from fixes

```

Environment Files

.env (Local Development)

```

FLASK_CONFIG=development
MYSQL_HOST=localhost
MYSQL_USER=root
MYSQL_PASSWORD=localpass
MYSQL_DB=sdll

```

.env.prod (Production Database Access)

```

# For polling production errors locally
MYSQL_HOST=your-prod-host.railway.app
MYSQL_PORT=12345

```

```
MYSQL_USER=root
MYSQL_PASSWORD=prodpassword
MYSQL_DB=railway
```

Safety Controls

Rate Limiting

Control	Default	Purpose
Max errors per hour	5	Pause if too many errors
Max diagnoses per day	10	Prevent runaway fixes
Cool-down between diagnoses	10 min	Time to review each fix
Max attempts per error	2	Don't retry failing fixes

Circuit Breaker

When limits are exceeded: 1. System creates errors/PAUSED lock file 2. Sends alert (if configured) 3. No more errors are exported until manually resumed

Error Filtering

Only diagnose errors that match ALL criteria:

```
# Tier I only (500 errors)
if error.tier != 1:
    return False
|
# Skip non-critical contexts
SKIP_CONTEXTS = {'tracking', 'analytics', 'page view'}
if error.context in SKIP_CONTEXTS:
    return False
|
# Skip bot requests
BOT_PATTERNS = ['bot', 'crawler', 'spider', 'curl']
if any(p in user_agent.lower() for p in BOT_PATTERNS):
    return False
|
# Skip static paths
SKIP_PATHS = ['/health', '/favicon', '/static/']
if any(path.startswith(p) for p in SKIP_PATHS):
    return False
```

Emergency Controls

Action	Command
--------	---------

PAUSE immediately	echo "paused" > errors\PAUSED
Resume	del errors\PAUSED (Windows) or rm errors/PAUSED
Check status	python scripts/poll_errors.py --status
Skip specific error	echo "skip" > errors\SKIP_123
Stop scheduled task	schtasks /end /tn "SDLLError Poll"

Part 4: TDD Fix Workflow

The Process

Every bug fix follows Test-Driven Development:

1. **Read Error** - Understand what happened
2. **Write Test** - Create test that reproduces the bug
3. **Verify Failure** - Run test, confirm it fails
4. **Implement Fix** - Make minimal code changes
5. **Verify Success** - Run test, confirm it passes
6. **Run Full Suite** - Ensure no regressions
7. **Get Approval** - Human reviews before commit
8. **Commit Together** - Fix AND test in same commit

Regression Test Template

```
# tests/test_regressions.py

class TestProductionRegressions:
    """Tests that reproduce and verify fixes for production errors."""

    def test_regression_error_42_null_field_name(self, app, field_factory):
        """
        Production Error: #42
        Context: Field display on schedule page
        Error: AttributeError: 'NoneType' object has no attribute 'name'
        Path: /schedule/2026/fall
        Root cause: Field was deleted but game still referenced it
        """
        with app.app_context():
            # 1. Create the conditions that caused the error
            field = field_factory('Test Field')
            game = Game(location_id=field.ID, ...)
            db.session.add(game)
            db.session.commit()

            # Delete field (simulating the bug condition)
            field.active = 0
            db.session.commit()

            # 2. Call the code that failed
            # Before fix: This would raise AttributeError
            # After fix: This should handle None gracefully
            result = get_game_location_name(game)

            # 3. Assert expected behavior
            assert result == "Unknown Field" # Graceful fallback
```

Claude Code Diagnosis Commands

```
# List pending errors
python scripts/diagnose_error.py

# View specific error
python scripts/diagnose_error.py 42

# Full diagnosis with Claude Code
claude "Diagnose and fix production error 42"

# Mark error as fixed
python scripts/diagnose_error.py --mark-fixed 42

# Mark error as skipped (won't retry)
python scripts/diagnose_error.py --mark-skipped 42 --reason "Expected behavior"
```

Part 5: Setup Guide

Initial Setup

1. Create Error Table

```
-- Run on production database
CREATE TABLE IF NOT EXISTS sdll_app_errors (
  id INT AUTO_INCREMENT PRIMARY KEY,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  tier SMALLINT NOT NULL DEFAULT 2,
  context VARCHAR(100) NOT NULL,
  error_type VARCHAR(100) NOT NULL,
  error_message TEXT NOT NULL,
  traceback TEXT,
  request_method VARCHAR(10),
  request_path VARCHAR(500),
  request_user_agent VARCHAR(500),
  user_id INT,
  notified BOOLEAN DEFAULT FALSE,
  resolved BOOLEAN DEFAULT FALSE,
  resolved_at DATETIME,
  resolved_by INT,
  error_hash VARCHAR(64),
  INDEX idx_tier (tier),
  INDEX idx_created_at (created_at),
  INDEX idx_resolved (resolved)
);
```

2. Register Global Error Handler

```
# app/__init__.py
from app.utils.errors import register_global_handler

def create_app(config_name=None):
    app = Flask(__name__)
    # ... other setup ...
    register_global_handler(app)
    return app
```

3. Create Production Credentials File

```
# .env.prod (add to .gitignore!)
MYSQL_HOST=your-prod-host
MYSQL_PORT=12345
MYSQL_USER=root
MYSQL_PASSWORD=yourpassword
MYSQL_DB=railway
```

4. Setup Polling Task (Windows)

```
scripts\setup_error_poll_task.bat
```

5. Setup Polling Task (Linux/Mac)

```
# Add to crontab
*/5 * * * * cd /path/to/project && python scripts/poll_errors.py
```

Daily Workflow

1. **Morning:** Check `python scripts/poll_errors.py --status`
2. **If errors pending:** Run claude `"Diagnose pending errors"`
3. **Review fixes:** Approve or reject Claude's proposed changes
4. **Deploy:** Push to production after tests pass

Appendix A: File Reference

Scripts

File	Purpose
<code>run_tests.py</code>	Main test runner with layer support
<code>scripts/poll_errors.py</code>	Poll production DB for errors
<code>scripts/diagnose_error.py</code>	View and manage error queue
<code>scripts/smoke_test.py</code>	Run full smoke test suite
<code>scripts/pre_push_check.py</code>	Pre-push validation

Configuration

File	Purpose
<code>.env</code>	Local development config
<code>.env.prod</code>	Production database access
<code>tests/inventory.yaml</code>	Feature inventory for smoke tests
<code>tests/conftest.py</code>	Pytest fixtures and factories

Error Handling

File	Purpose
app/utils/errors.py	Error logging utilities
app/models/app_error.py	AppError database model
app/services/error_diagnosis_service.py	Error export service

Appendix B: Troubleshooting

Common Issues

"Table doesn't exist"

Run the migration SQL on your production database.

Poll script times out

Check .env.prod has correct host, port, and credentials.

Tests fail with "No database"

Ensure MySQL is running and .env has correct local credentials.

Claude Code can't find files

Run from project root directory.

Getting Help

- Check `python scripts/poll_errors.py --status`
- View recent errors: `python scripts/diagnose_error.py`
- Check logs in `errors/` directory